BATON: Runtime Handover Between Specialist Policies

The switch is the engineering.

A. Fetouh
OmniSim Research
Technical Report — July 2026

Abstract

The standard way to give one robot many skills is to train *one* policy on all of them and condition it on a command. The alternative—keep independently-trained **specialists** and switch between them—is usually dismissed, because the switch is where it breaks. This report is about the switch. We describe **BATON**, an engineered handover: a morph blend, a phase-gated entry, and a **recurrent-state law** (a warm LSTM state carried out of a stand into a locomotion policy locks the incoming policy in the stand attractor; it marches in place). We package the handover as data. ■ **THE HORIZON EXPERIMENT HAS NOT BEEN RUN ON THIS CHECKOUT.** This build has no results section, by construction: the figures and numbers are read from a results file that does not exist, and this document will not print a number it does not have. Run `scripts/dev/baton_horizon_experiment.sh` and rebuild.

1 The switch, not the skills

A humanoid that must walk, stop, turn, carry and climb is usually built as one velocity-conditioned policy: a single network trained on every skill, told which one to do by a command vector. It works, and it has two costs that grow with the skill set. Every skill is trained against the interference of every other skill, so the marginal skill makes the others slightly worse; and a genuinely new skill—one that does not live on the same continuum, like a carry or a get-up—cannot be added without retraining the whole thing.

Keeping separate specialists removes both costs and creates exactly one new problem: at the moment of the switch, a policy that has never seen this state is handed a robot in it. The literature that reports long-horizon numbers reports them degrading—a recent large-humanoid-model result falls from 90% to 18% success over five cycles of a repeated task—and the hierarchical baselines it is compared against hand over with a naive finite-state machine: swap the network, keep going. Our claim is narrow and, we think, useful: **most of the difficulty of policy switching is in the handover, and the handover is engineerable.**

2 What breaks at a handover

Four things, and only the first is obvious.

(i) The command discontinuity. The outgoing policy was tracking a 0.45 m/s gait; the incoming one wants 0. Stepping the command in one tick is a shove. We ramp it (a *morph* blend over N ticks).

(ii) The phase discontinuity. A cyclic specialist is a function of gait phase. Entering it at an arbitrary phase asks it to continue a stride it never started. We gate entry on a compatible phase.

(iii) The recurrent state. This is the one that cost us the most, and it is the reason we think handover is a research object and not a plumbing detail. Our specialists are LSTMs. The hidden state is not a detail of the implementation: it *is* the policy's belief about what it is doing. Hand a walker the hidden state of a policy that has just spent three seconds standing still, and the walker does not walk—it **marches in place**, locked in the stand attractor, because its own recurrent state is telling it that it is standing. The fix is not a bigger network. It is a *law*: at a `stand` → `locomotion` edge, the hidden state is reset (**cold**); at a `locomotion` → `locomotion` edge it is carried (**warm**), because there the belief is true and worth keeping. In our library this is derived from the skills' declared attractors, not configured per demo.

(iv) The context. A specialist carries more than weights: a reference (its ghost), an observation layout, a corridor, a clock. Two specialists in the same observation family can be blended element-wise. One with a different observation width cannot be blended at all, and must instead have its whole deploy context *swapped* for the duration of its segment. Both

mechanisms exist in our runtime; the manifest decides which applies.

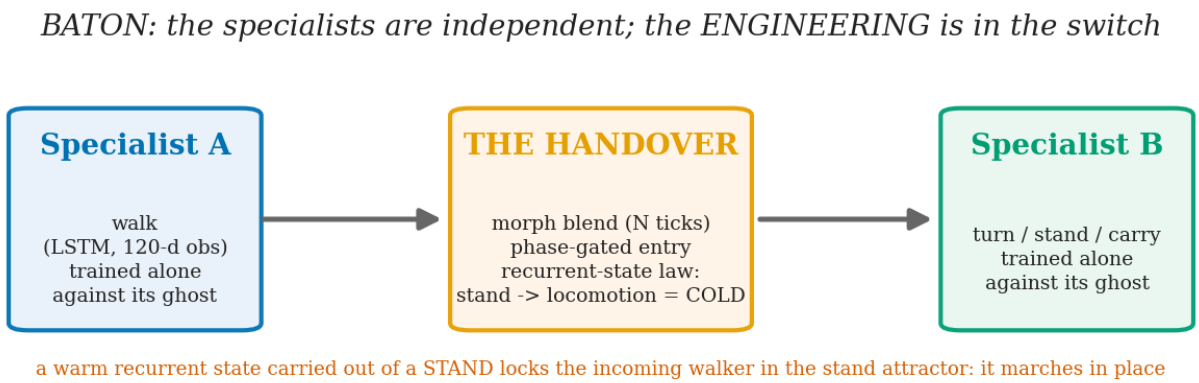

Figure 1: BATON. The specialists are trained independently against their own references; the engineering is in the switch between them.

3 The mechanism

A **skill** is a manifest: a reference (a phase-indexed lookup table), a policy checkpoint, an observation family, a deploy context, a BATON *blend* (`cyclic` or `solo_swap`), and an *attractor* (`stand` or `locomotion`). A **sequence** is a list of skills plus an arbiter (a schedule, or a goal-directed course of waypoints). The handover law is *derived* from the manifests—the cold/warm decision at each edge falls out of the attractors, and is not a knob a demo author sets by hand. Sequences are checked against the hand-written demo scripts they replace, key-for-key on the assembled launch environment.

The specialists themselves are produced by *Shadowing* (companion report): each tracks a reference that was certified feasible before training. That matters here only in that it makes the specialists comparable—they share a training recipe, an observation family and a corridor, so a handover between them is a fair test of the handover rather than a test of two unrelated networks.

4 The turn, and why a corner is not a segment

The hardest handover we ship is the 90° corner. A turn specialist trained on a step-turn reference banks only ~60–65% of its reference yaw when replayed once on the real robot: the feet track the reference but the base under-rotates (foot slip is per-step, not per-second, so a slower clock cannot add rotation—more *steps* can). The reference is a modular staircase of mini-pivots, each beginning and ending feet-together, so we replay *partial passes*, restarting at the plateau whose remaining staircase matches the remaining angle divided by the measured gain, until the **actual accumulated heading** reaches the target. Never stopping mid-reference is a hard rule: the robot lags the reference by one to two mini-cycles, so a reference plateau says nothing about the robot's stance, and both mid-reference arrests we measured recoiled or spun.

5 Experiments

5.1 The shipped sequences

Four sequences ship as manifests and reproduce their hand-written demos exactly: a box delivery (walk→stand→pick→carry→90° corner→place→walk), its corner-free variant, a walk→turn→walk, and a solo turn. The 90° corner lands at 90.6 / 91.5 / 95.6° actual heading, 3/3, zero falls, with *zero crane yaw torque*—the rotation is the robot's own footwork.

5.2 Success versus horizon

■ **NOT YET RUN ON THIS CHECKOUT.** This section is generated from `_scratch/baton_horizon/results.json`, which does not exist here. The build refuses to print a horizon curve, a table or a claim it does not have. Run `scripts/dev/baton_horizon_experiment.sh 6 5 900` and rebuild.

6 What we do not claim

We have not beaten a monolith. The third arm of the comparison—a single policy trained honestly on the same skill set with a real budget—is a training campaign, not a deploy sweep, and we have not run it. Everything above compares an engineered handover to a naive one; it says the switch is worth engineering, and it says nothing about whether specialists-plus-switching beats one big conditioned network. The literature's own hierarchical baselines are naive-FSM, which is precisely the arm we beat, so it would be easy and dishonest to slide from one claim to the other. **'Switching beats a monolith' remains an open hypothesis**, and the experiment that would settle it is specified in our notes.

The humanoid is on a harness. Every result here runs on a weight-bearing balance crane that carries up to about twice the robot's weight and holds its attitude. The legs step and the corner is genuine footwork (zero crane yaw torque during the turn), but the robot is being *carried*. A durable free-standing humanoid walk remains an open problem in this system, and no number in this report should be read as one.

Blending requires a shared observation family. Cyclic peers are blended element-wise, which requires identical observation widths; a specialist that does not share the family can only be context-swapped for its whole segment. Cross-*robot* handover is not a goal and is not supported.

7 Conclusion

Policy switching is usually treated as plumbing between the interesting parts. We think it is the interesting part. A handover has a command discontinuity, a phase discontinuity, a context, and—if the specialists are recurrent—a *belief* that must be invalidated at exactly the right edges. Getting those right is what separates a chain of specialists that survives a long task from one that marches in place at the first corner. We have made the handover explicit, made it data, and measured it against the naive alternative. The larger question—whether this architecture beats the monolith it is meant to replace—is still open, and we would rather say so than imply otherwise.